

Lab #2

Introduction to compiling and debugging a C program using the MPLAB

ECE 3301L Introduction to Microcontrollers Laboratory

Sarah Boyd

6/22/25

Dr. Tamer Omar



**Cal Poly
Pomona**

College of Engineering

Execution videos and program files:

Prelab i:

- Output/ analysis video:
<https://youtu.be/cX5sMVFTn3U>

Prelab ii:

- Output/analysis video:
<https://youtu.be/7tzeveD4HM0>

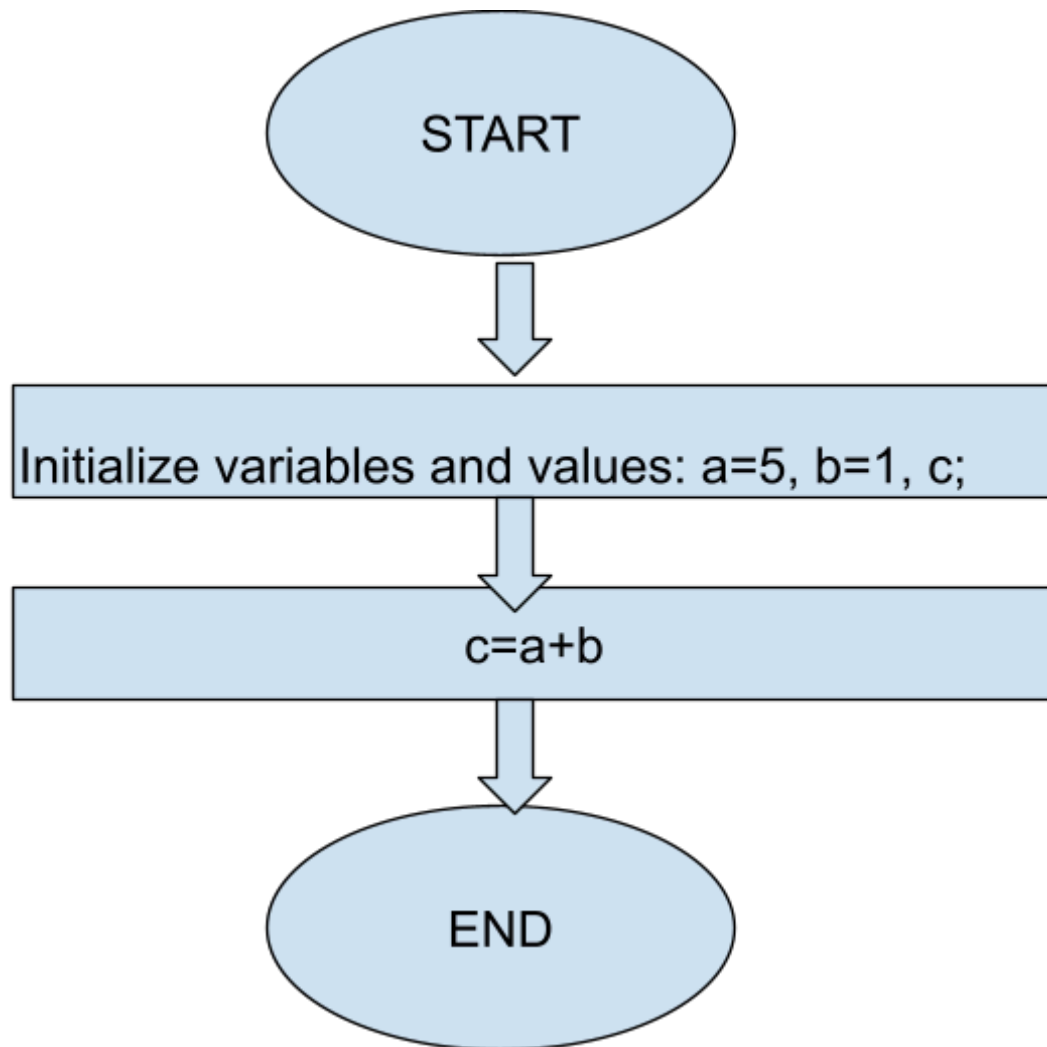
Postlab

- Output/analysis video:
<https://youtu.be/xrnkgY6yXTI>

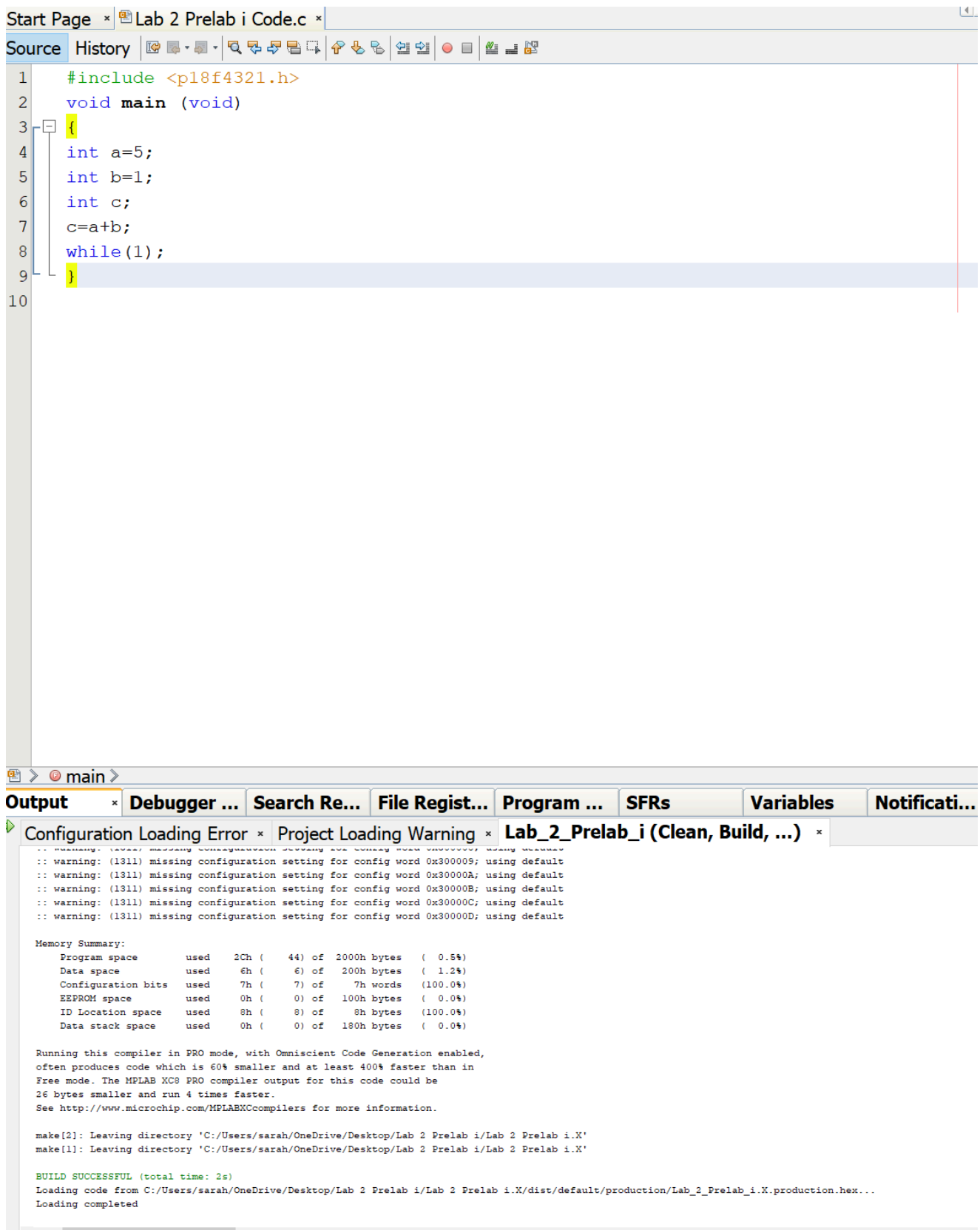
Objective: The objective of this lab is to compile and debug a C program using MPLAB C18 or XC8 compiler, to add two 16-bit numbers, each containing two ASCII numbers, and then store the result as a packed BCD byte.

PRELAB:

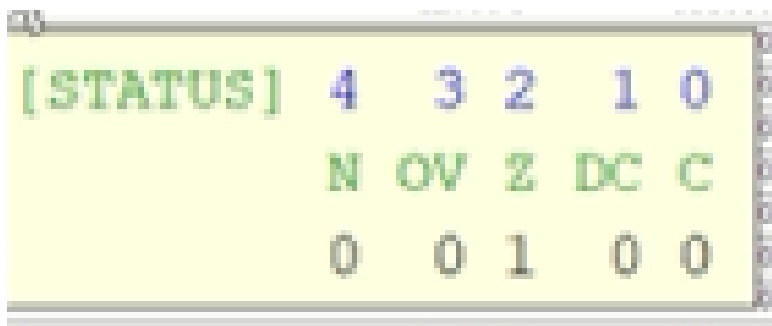
prelab i) Compile and execute the tutorial program written in C language using the MPLAB C18 or XC8 Compiler (Appendix G of the book)



IMG 1. Flow chart



IMG 2. Code from appendix G of the book, built successfully.



IMG 3. Status values

Output	Debugger Console								Search Results					File Registers ×					Pro
	Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII	
	000	00	06	00	05	00	01	00	00	00	00	00	00	00	00	00	00		
	010	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
	020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
	030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
	040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
	050	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
	060	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
	070	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		
	080	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00		

IMG 4. File register values

Name	Type	Address	Value
☒ a	int	0x3	0x0005
☒ b	int	0x5	0x0001
☒ c	int	0x1	0x0006
<Enter new watch>			
☒ a	int	0x3	0x0005
☒ b	int	0x5	0x0001
☒ c	int	0x1	0x0006

IMG 5. Values of variables c, a and b

prelab ii)

Write a C language program to add two 16-bit numbers by converting each number from ASCII number representation into packed a BCD byte

1) First 2 digit ASCII number 0x3735 --> Convert it to unpacked BCD 0x0705 --> Convert it to packed BCD 0x75

2) Second 2 digit ASCII number 0x3232 --> Convert it to unpacked BCD 0x0202 → Convert it to packed BCD 0x22

3) Use the variable Result to add the two packed BCD numbers. Result = 75H + 22H = 97H

Conceptual background and explanation of code:

ASCII (American Standard Code for Information Interchange) is a way that computers represent characters (such as letters, digits and symbols) as numbers. The digit 0 is represented as 0x30, so the digit 1 would be written as 0x31, the digit 2 as 0x32, and so on. Numbers with 16 bits (or 2 bytes) such as 0x3735 are like 0x37 0x35, or 75. BCD (binary coded decimal) is a way to store numbers in binary. The whole number isn't stored in binary, but each decimal digit is stored in its own binary number. For example, if we wanted to write the decimal digit 75 in binary, it would be written as 01001011. Our first operand, 0x3735, looks like 0011 0111 0011 0101 in binary. The high byte is 0011 0111 (0x37) while the low byte is 0011 0101 (0x35)

Goal = 75	0	+	64	+	0	+	0	+	8	+	0	+	2	+	1
Place	$2^7 = 128$		$2^6 = 64$		$2^5 = 32$		$2^4 = 16$		$2^3 = 8$		$2^2 = 4$		$2^1 = 2$		$2^0 = 1$
Bit	0		1		0		0		1		0		1		1

Table 1. 75 in binary. This chart is applicable for the concept of bit compared to decimal value.

For this program, we need to separate the high and low 8 bit parts. We can get the high byte by moving the high byte (the left 8 bits) into the lower byte position by doing `(Op1 >> 8)`. This turns our original phrase of 0011 0111 0011 0101 into 0000 0000 0011 0111. We add an “0xFF” to only keep the bottom 8 bits, and to make sure the first 8 bits are cleared to 0. We can create a new variable to represent the high byte of operand one and have it now equal 0011 0111 or 0x37. So all together, we get the line `uint8_t Op1_highByte = (Op1 >> 8) & 0xFF`. This takes the top 8 bits (or the first character) from the first 16-bit ASCII number (Op1) and stores them as an 8 bit variable to later use to get the digit for the 10s place.

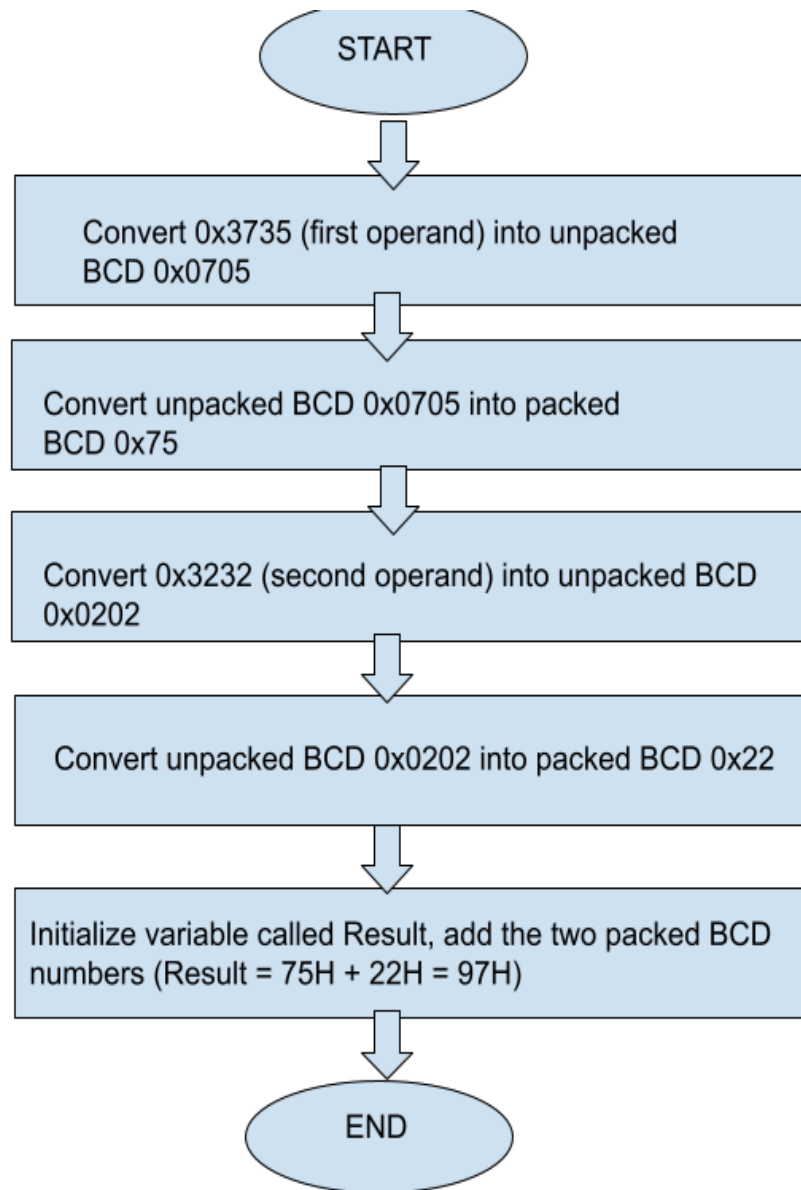
To get the low byte, we need to extract the last 8 bits from Op1, get rid of everything else and then set it equal to a variable. We can do this by doing `uint8_t Op1_lowByte = Op1 & 0xFF`. This takes the lower 8 bits (the second character) from the first operand and saves it as a separate variable we can later use to get the digit for the 1s place.

Now we have a 10s value of 0x37 and a 1's value of 0x35. We can subtract 0x30 from both and get a 10s value of 0x07 and a 1's value of 0x05. So we have our unpacked BCD digits- 0x07 and 0x05.

To get packed BCD digits, we want to combine the 10s digit (upper nibble) and 1s digit (lower nibble). The 10s digit is currently 0x07 (or 0000 0111), so we shift the remaining 4 bits by `0x07 << 4` or `Op1_tens << 4`, which gives us 0111 0000 or 0x70. To make sure we are only taking the last 4 bits of the ones value, we can do `Op1_ones & 0x0F` to get 0x05. Then we can add the 10s and 1s, `0x70 + 0x05` or `0111 0000 + 0000 0101` and get a packed BCD of 0x75.

We then repeat the same process for the second operand and then add the two packed BCD numbers, storing it in the Result variable.

Code design:



IMG 6. Flow chart

Start Page Lab 2 Prelab ii Code.C

Source History

```

8  uint16_t Op1 = 0x3735; // Operand 1
9  uint16_t Op2 = 0x3232; // Operand 2
10
11 // Then, we extract the individual ASCII characters from operand 1
12 //Extract the the high byte to get the ACII for the 10s digit
13 uint8_t Op1_highByte = (Op1 >> 8) & 0xFF; // shift right 8 bits to get the high byte and make sure we only keep the last byte by setting the first
14 //Extract the low byte to get the ACII for the 1s digit
15 uint8_t Op1_lowByte = Op1 & 0xFF; // 0x35 (ASCII for 5)
16
17 // Convert ASCII to unpacked BCD
18 uint8_t Op1_tens = Op1_highByte - 0x30; // 0x07
19 uint8_t Op1_ones = Op1_lowByte - 0x30; // 0x05
20
21 // Convert unpacked to packed BCD for Operand 1
22 uint8_t Op1_packed = (Op1_tens << 4) | (Op1_ones & 0x0F); // 0x70 + 0x05 = 0x75
23
24 //Next, we extract the individual ASCII characters for operand 2
25 //We first extract the high byte to later get the 10s digit
26 uint8_t Op2_highByte = (Op2 >> 8) & 0xFF; // 0x32 (ASCII for '2')
27 //Then we extract the low byte to get the ACII for the 1s digit
28 uint8_t Op2_lowByte = Op2 & 0xFF; // 0x32 (ASCII for '2')
29
30 // Convert ASCII to unpacked BCD
31 uint8_t Op2_tens = Op2_highByte - 0x30; // 0x02
32 uint8_t Op2_ones = Op2_lowByte - 0x30; // 0x02
33
34 // Convert unpacked to packed BCD for Operand 2
35 uint8_t Op2_packed = (Op2_tens << 4) | (Op2_ones & 0x0F); // 0x20 + 0x02 = x22
36
37 // Add the two packed BCD numbers and store in Result
38 uint8_t Result = Op1_packed + Op2_packed; // 0x75 + 0x22 = 0x97
39
40 }
41
42

```

main

Output Debugger Console Search Results File Registers Program Memory SFRs Variables Notifications Call Stack

Name	Type	Address	Value
☒ Op1	unsigned int	0xD	0x3735
☒ Op1_highByte	unsigned char	0x3	'7'; 0x37
☒ Op1_lowByte	unsigned char	0x4	'5'; 0x35
☒ Op2	unsigned int	0xF	0x3232
☒ Op2_highByte	unsigned char	0x8	'2'; 0x32
☒ Op2_lowByte	unsigned char	0x9	'2'; 0x32
☒ Op1_tens	unsigned char	0x5	BEL; 0x7
☒ Op1_ones	unsigned char	0x6	ENQ; 0x5
☒ Op2_tens	unsigned char	0xA	STX; 0x2
☒ Op2_ones	unsigned char	0xB	STX; 0x2
☒ Op1_packed	unsigned char	0x7	'u'; 0x75
☒ Op2_packed	unsigned char	0xC	""; 0x22
☒ Result	unsigned char	0x2	'?'; 0x97
☐ <Enter new watch>			

Lab 2 Prelab ii (Build, Load, ...)

IMG 7 . Working code with correct variable values. For a line-by-line explanation of the code and more details concerning the variables and STATUS output, please see the video explaining the code. Additionally, the comments on the submitted code file explain the code line-by line.

4. Equipment, Software and Components required:

Microchip's MPLAB C18 or XC8 Compiler /Debugger

5.Description (corresponding topics covered in the textbook)

ASCII numbers are 8-bit numbers with the high 4 bits as 00112 or 310. These numbers can be converted to packed BCD by masking off the high 4 bits. Since the numbers to be added are 16-bit wide, one can convert the numbers into 8-bit packed BCD by masking off the high 4 bits of each ASCII number. The high byte can be shifted 4 times to the left and logically ORed with the low byte. The two converted 8-bit packed BCD numbers can then be added.

6.Prerequisites:

Sections 1.2. Microcontroller Data Types

7. Procedure:

Compile, debug, and verify the C-program written for Prelab 2

POSTLAB:

Write a PIC18F assembly language program to accomplish the above.

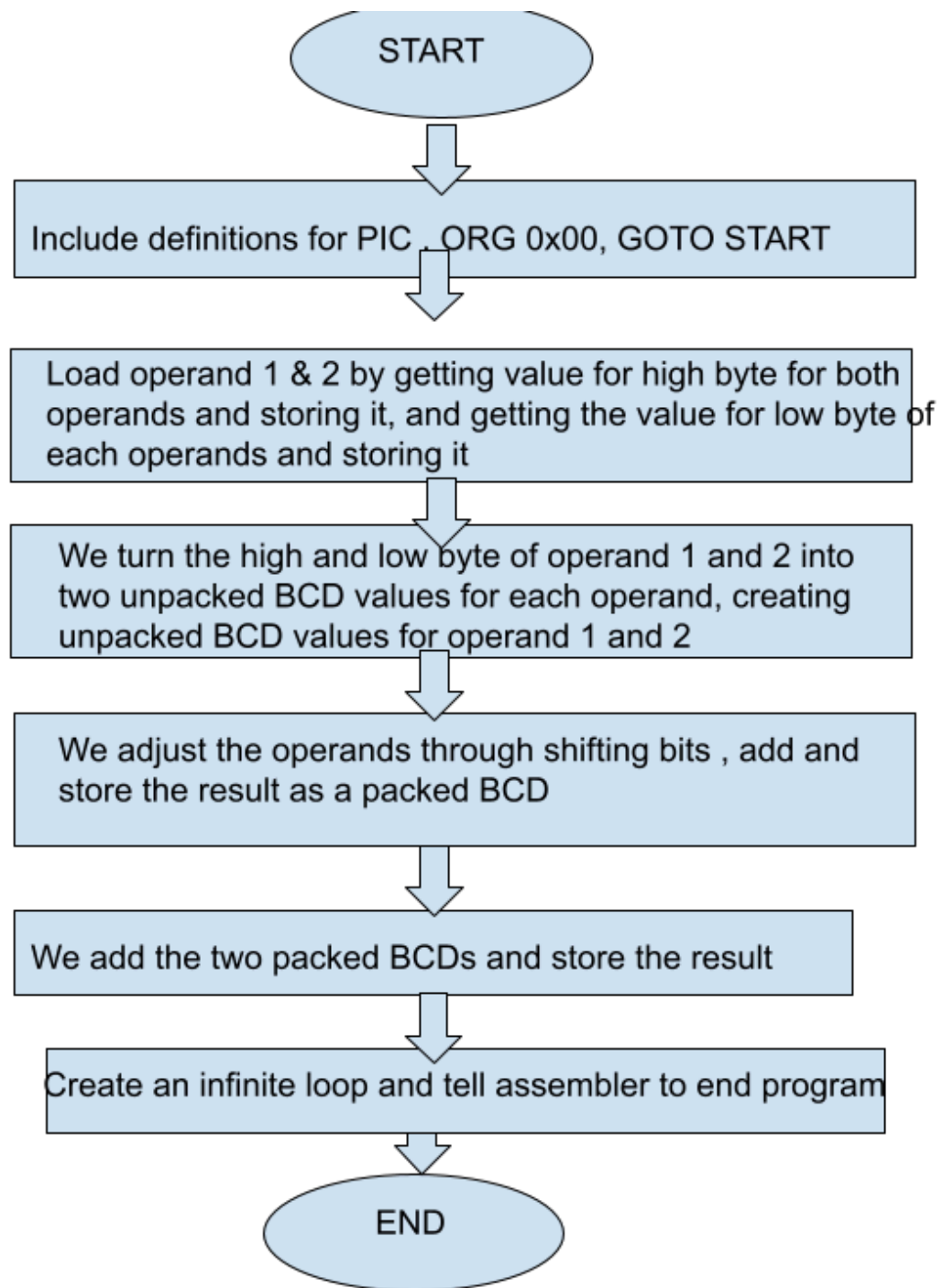
Conceptual background and explanation of code:

Again, we are given the two 16-bit ASCII values 0x3735 and 0x3232, with the same goal as the code in prelab ii.

First, we need to determine the meaning of the file register memory locations, and what each memory location should store.

Variable in my C code	File register location in my assembly code	What it will store
Op1	0x20, 0x21	The first 16 bit operand, 0x3735. 0x20 is for the low byte and 0x21 is for the high byte.
Op2	0x22, 0x23	The second 16 bit operand, 0x3232. 0x22 is where we store the low byte and 0x23 is where we store the high byte
Op1_tens	0x24	The high digit of operand 1 before adjusting (0x37-0x30 =0x07)
Op1_ones	0x25	The low digit of operand 1 (0x35-0x30 =0x05)
Op1_packed	0x26	The packed BCD of 75
Op2_tens	0x27	The high digit of operand 2 before adjusting (0x32 -0x30 =0x02)
Op2_ones	0x28	The low digit of operand 2 (0x32-0x30 =0x02)
Op2_packed	0x29	The packed BCD of 22
Result	0x2A	The result (0x75 + 0x22 =0x97)

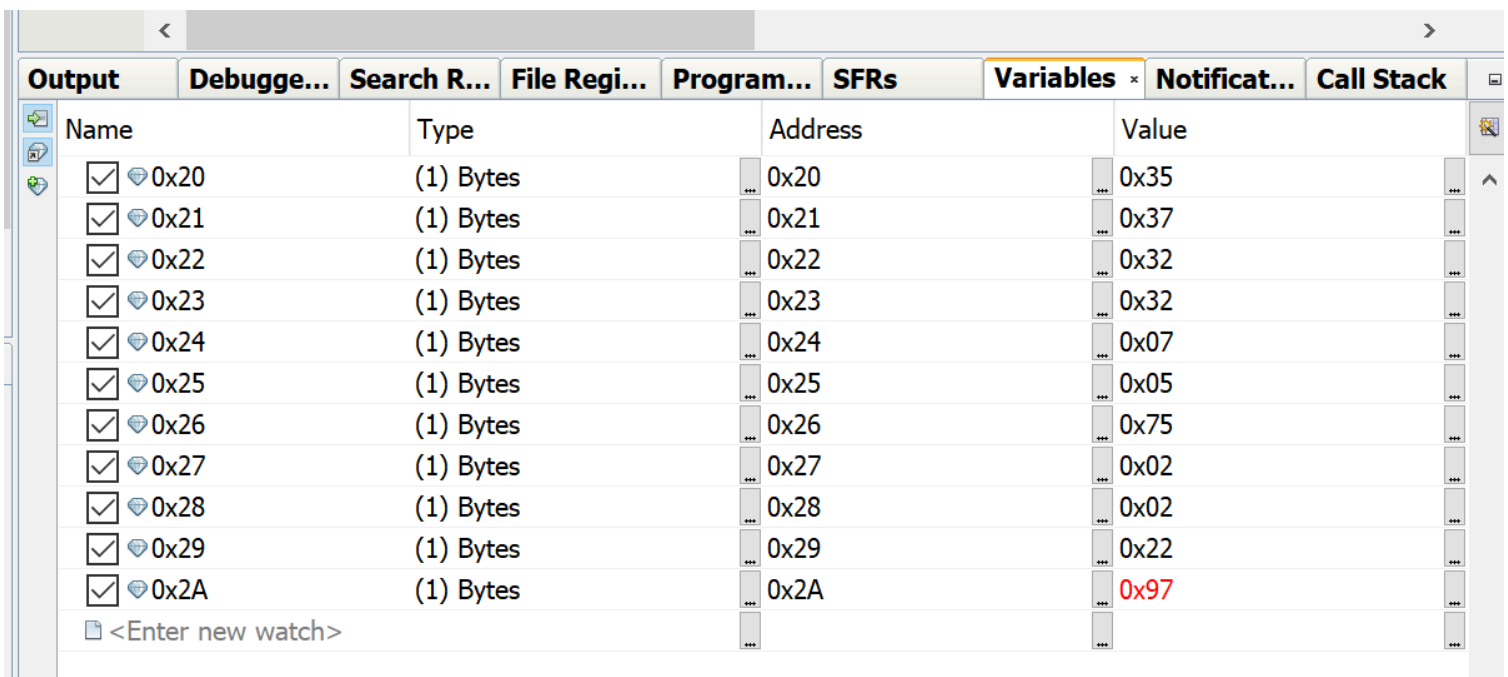
Table 2. Our previously used variables in our C code, the memory location storing the data that was stored in these C variables, and explanation behind what the value in each memory location represents.



IMG 8 . Flow chart

MOVLW	Move literal value into WREG
MOVWF	Move WREG into file register
MOVF	Move value from the file register into WREG
SWAPF	Swap upper and lower 4 bits of a byte
ANDLW	Mask off bits you don't want, and used to keep just part of a byte (the high nibble or low nibble)
IORWF	Combines two binary values (the value in the WREG register and the value in a file register) and then stores result back in file register if “, F” or back into WREG if there's a “,W” at the end
SUBWF	Subtracts WREG a file register.
ADDWF	Adds file register to WREG
GOTO	Jump to label (START)

Table 2. Key terms for this code



Name	Type	Address	Value
☒ 0x20	(1) Bytes	0x20	0x35
☒ 0x21	(1) Bytes	0x21	0x37
☒ 0x22	(1) Bytes	0x22	0x32
☒ 0x23	(1) Bytes	0x23	0x32
☒ 0x24	(1) Bytes	0x24	0x07
☒ 0x25	(1) Bytes	0x25	0x05
☒ 0x26	(1) Bytes	0x26	0x75
☒ 0x27	(1) Bytes	0x27	0x02
☒ 0x28	(1) Bytes	0x28	0x02
☒ 0x29	(1) Bytes	0x29	0x22
☒ 0x2A	(1) Bytes	0x2A	0x97
<Enter new watch>			

IMG 10. Actual values of key memory locations in this program. Note that the actual results are identical to the predicted and intended results in table 2. For more information, please see the video explaining the code linked on page 2.